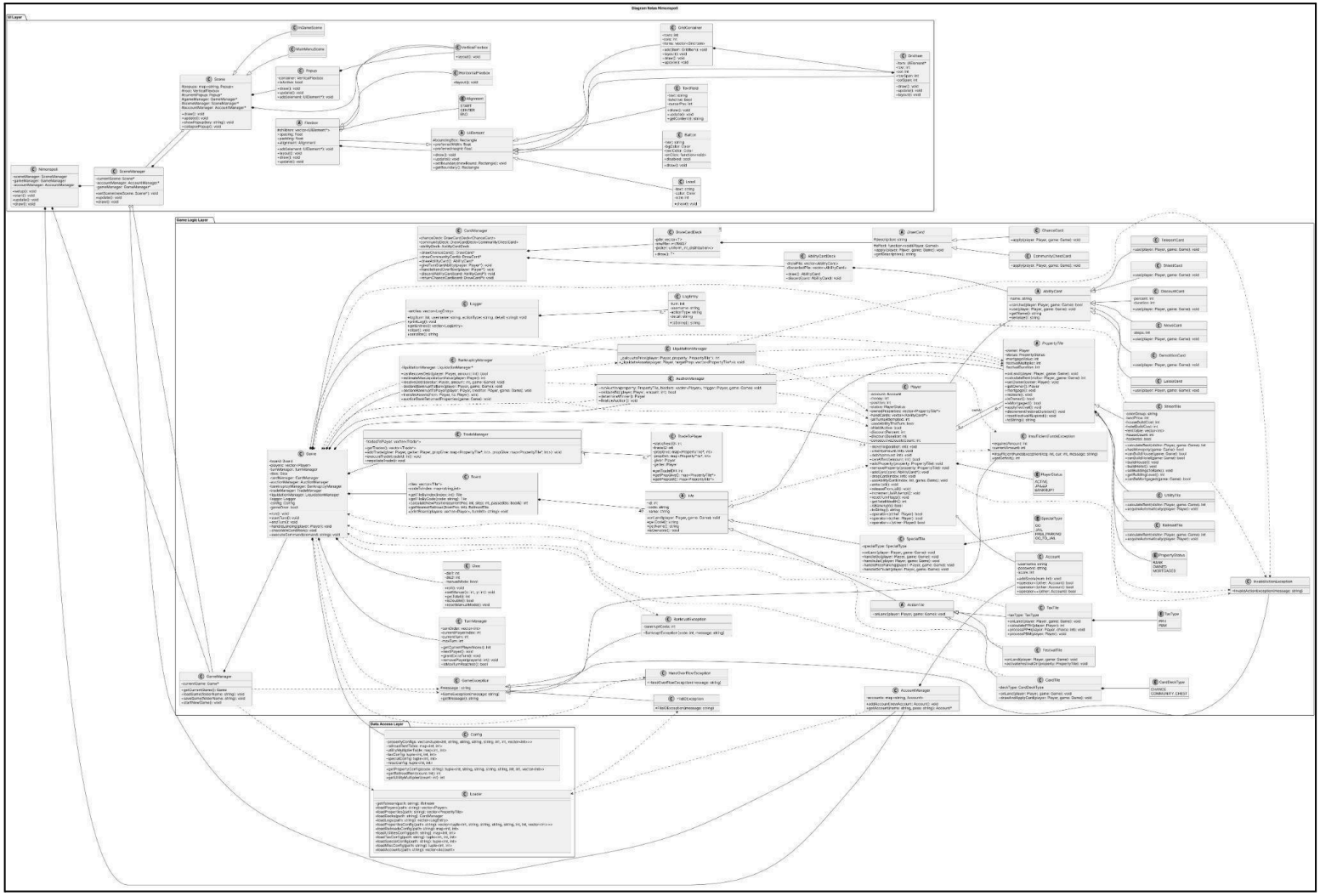
Laporan M1 Tugas Besar 1
IF2010 Pemrograman Berorientasi Objek 2025/2026
Nimonspoli

Kode Kelompok : SWT
Nama Kelompok : SWIT

1. 13524017 / Aziza Dharma Putri
2. 13524029 / Niko Samuel Simanjuntak
3. 13524045 / Ahmad Zaky Robbani
4. 13524063 / Marcel Luther Sitorus
5. 13524081 / Alya Nur Rahmah

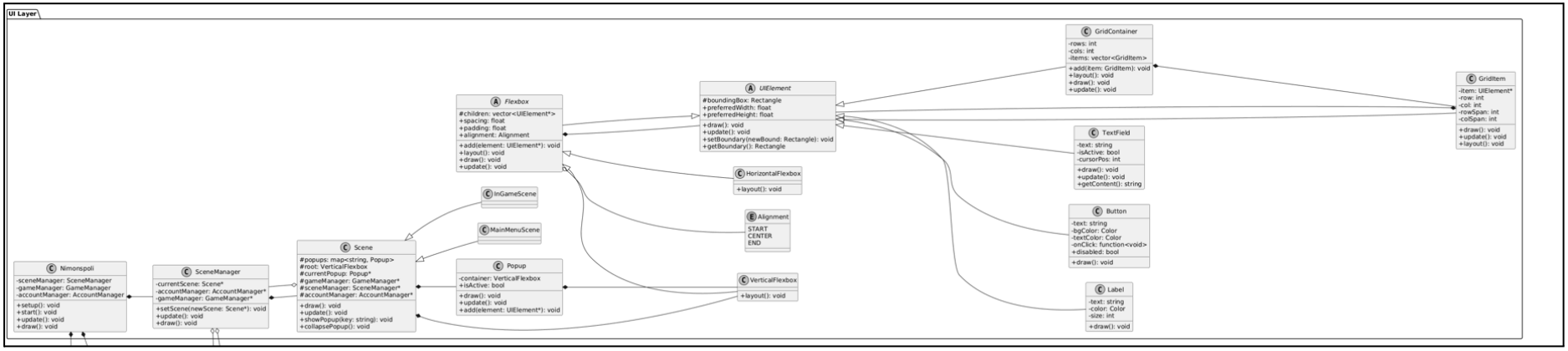
SWT - SWIT

1. Diagram Kelas

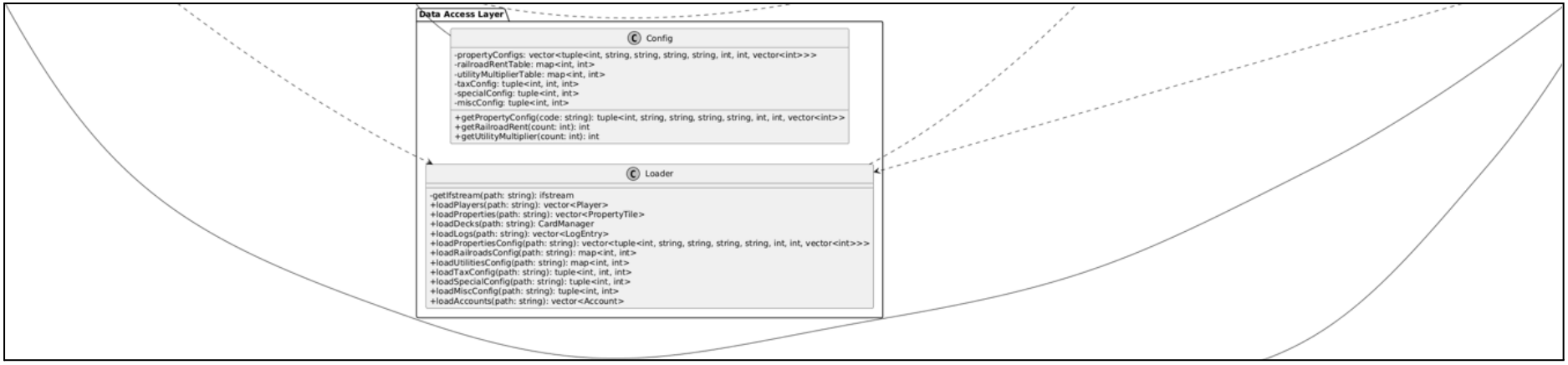


Gambar 1.1 Diagram Kelas

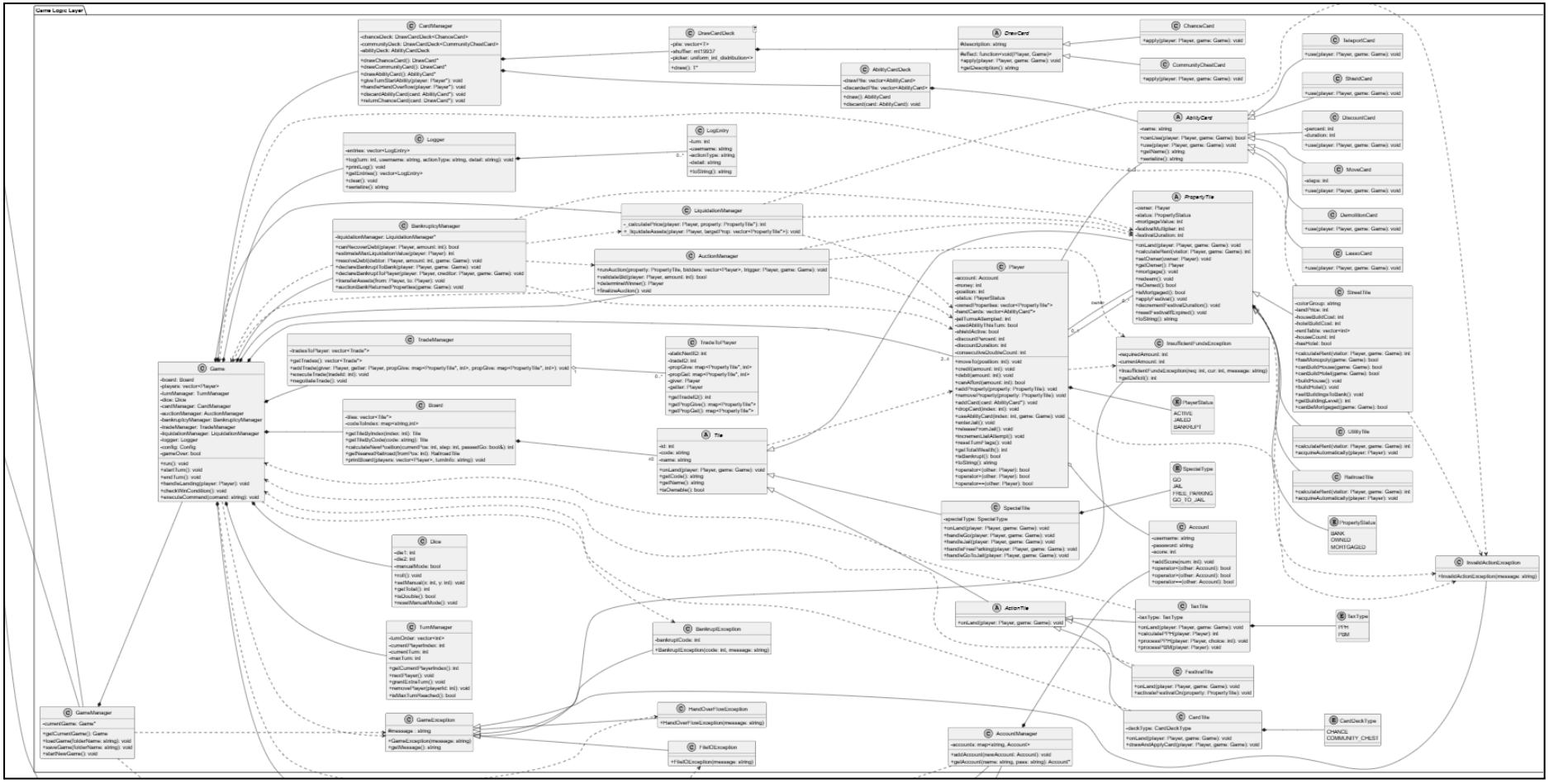
Diagram kelas pada sistem ini dibuat menggunakan PlantUML, yaitu tools berbasis teks yang digunakan untuk menggambar diagram UML secara otomatis. Dengan PlantUML, diagram tidak digambar secara manual, melainkan ditulis dalam bentuk kode menggunakan sintaks tertentu dan berikut ini kode untuk diagram sistem Nimonspoli yang kami rancang bisa akses di sini. Untuk melihat preview diagram tersebut di VSCode, pastikan extension PlantUML sudah terpasang dan file disimpan dengan ekstensi .plantuml. Setelah itu, buka file tersebut lalu klik kanan di area editor dan pilih "Preview Current Diagram", atau gunakan shortcut Alt + D. Diagram akan langsung ditampilkan pada panel preview di samping editor. Untuk melihat tampilan diagram yang lebih jelas tanpa harus set-up dapat mengakses link berikut ini. Diagram yang kami buat dipisahkan berdasarkan layernya sebagai berikut :



Gambar 1.2 Diagram Kelas *UI Layer*



Gambar 1.3 Diagram Kelas *Data Access Layer*



Gambar 1.4 Diagram Kelas *Game Logic Layer*

2. Deskripsi Kelas

2.1. Kelas Nimonspoli

Kelas utama yang menjadi titik masuk seluruh aplikasi. Bertanggung jawab menginisialisasi dan mengorkestrasi tiga manajer utama: SceneManager (tampilan), GameManager (logika permainan), dan AccountManager (akun pemain).

Atribut

- sceneManager : Mengelola perpindahan dan rendering scene (MainMenu, InGame)
- gameManager : Mengelola state permainan aktif dan persistensi (save/load)
- accountManager : Mengelola akun dan leaderboard pemain

Method

- setup() : Inisialisasi semua manajer, load konfigurasi, siapkan scene awal
- start() : Menjalankan game loop utama
- update() : Mendelegasikan update ke SceneManager setiap frame
- draw() : Mendelegasikan rendering ke SceneManager setiap frame

Hubungan

- Komposisi dengan Kelas GameManager, AccountManager, SceneManager

2.2. Kelas UIElement

Abstract base untuk semua komponen UI visual. Mendefinisikan kontrak bahwa setiap elemen UI harus bisa di-draw, di-update, dan memiliki bounding box untuk layout. Memungkinkan container (Flexbox, GridContainer) menyimpan campuran elemen UI dalam satu koleksi.

Atribut

- boundingBox : Rectangle area yang dimiliki elemen ini di layar
- preferredWidth/Heigh : Ukuran yang diminta elemen untuk digunakan parent container saat layout

Method

- draw() : Abstract render elemen ke layar sesuai boundingBox
- update() : Abstract proses input dan update state elemen
- setBoundary/getBoundary() : Dipanggil container saat melakukan layout pass

Hubungan
- Inheritence Label, Button, TextField, GridItem, GridContainer, Flexbox

2.3. Kelas Label

Elemen UI paling dasar untuk menampilkan teks statis. Digunakan di seluruh Scene untuk judul, label form, informasi properti, status pemain, dan teks informatif lainnya. Tidak menerima input dari user.

Atribut
- text : Konten teks yang ditampilkanf
- color : Warna teks bertugas untuk memungkinkan Label menjadi berwarna untuk kode warna properti
- size : Ukuran font dalam piksel

Method
- draw() : Render teks di posisi boundingBox dengan color dan size yang diset

Hubungan
- Inheritence dengan Kelas UIElement

2.4. Kelas Button

Elemen UI yang bisa diklik. Menyimpan callback onClick sebagai function<void()> sehingga setiap tombol bisa dikonfigurasi aksinya tanpa subclass baru. Properti disabled mencegah interaksi saat tombol tidak relevan.

Atribut
- text : Teks label tombol
- bgColor, textColor : Warna visual tombol, bisa berbeda untuk tombol aktif vs disabled
- onClick : Lambda yang dieksekusi saat tombol ditekan, ini berisi aksi spesifik
- disabled : Jika bernilai True maka tombol tidak bisa diklik, tampil abu-abu

Method
- draw() : Render kotak tombol dengan teks, beda tampilan jika disabled

Hubungan
- Inheritence dengan Kelas UIElement

2.5. Kelas TextField

Elemen input teks dari user. Digunakan untuk form username/password di login/register, input perintah CLI, dan semua input teks lainnya. Mengelola state aktif/tidak aktif dan posisi kursor.

Atribut
- text : Konten yang sudah diketik user
- isActive : True jika field sedang fokus dan menerima keyboard input
- cursorPos : Posisi kursor dalam teks untuk insert/delete karakter

Method
- draw() : Render kotak input dengan teks dan kursor berkedip jika aktif
- update() : Proses keyboard event saat isActive bertugas tambah/hapus karakter, gerak kursor
- getContent() : Return teks saat ini, dipanggil Scene saat user submit input

Hubungan
- Inheritence dengan Kelas UIElement

2.6. Kelas GridItem

Wrapper yang menempatkan satu UIElement dalam sel tertentu di GridContainer. Menyimpan informasi posisi (row, col) dan span sehingga GridContainer bisa melakukan layout tanpa tahu konten dari masing-masing sel.

Atribut
- item : UIElement* yang ditempatkan dalam sel ini
- row, col : Posisi sel dalam grid (0-indexed)
- rowSpan, colSpan : Berapa sel yang ditempati untuk elemen yang lebih besar

Method
- draw() : Delegasikan ke item->draw()
- update() : Delegasikan ke item->update()

- layout() : Hitung boundingBox item berdasarkan posisi dan span dalam grid parent

Hubungan
- Inheritence dengan Kelas UIElement
- Komposisi dengan Kelas UIElement

2.7. Kelas GridContainer

Container yang menyusun elemen dalam grid baris-kolom. Berguna untuk tampilan papan permainan, form dengan label dan input sejajar, atau daftar properti dengan kolom terstruktur.

Atribut
- rows, cols : Dimensi grid
- items : Semua GridItem yang telah ditambahkan

Method
- add(item) : Tambahkan GridItem ke daftar
- layout() : Hitung ukuran tiap sel berdasarkan boundingBox container, set ke semua GridItem
- draw() : Loop items, delegasikan draw ke tiap GridItem
- update() : Loop items, delegasikan update ke tiap GridItem

Hubungan
- Inheritence dengan Kelas UIElement
- Komposisi dengan Kelas GridItem

2.8. Kelas Flexbox

Abstract container yang menyusun children secara linear. Dua implementasi konkret: VerticalFlexbox (tumpuk ke bawah) dan HorizontalFlexbox (jejer ke kanan). Spacing, padding, dan alignment dikonfigurasi di sini.

Atribut
- children : Daftar UIElement* yang dikelola
  spacing : Jarak antar children
- padding : Jarak dari tepi container ke children

- alignment : START/CENTER/END merupakan alignment di sumbu silang

Method
- add(element) : Tambah child ke daftar
- layout() : Abstract bertugas untuk menghitung boundingBox tiap child berdasarkan arah layout
- draw() : Loop children, delegasikan draw
- update() : Loop children, delegasikan update

Hubungan
- Inheritence dengan Kelas UIElement
- Inheritence dengan Kelas VerticalFlexbox, HorizontalFlexbox
- Komposisi dengan Kelas UIElement

2.9. Kelas VerticalFlexbox

Implementasi Flexbox yang menyusun children dari atas ke bawah. Setiap child diberi boundingBox dengan y yang digeser sesuai tinggi child sebelumnya ditambah spacing. Digunakan sebagai root container Scene dan isi Popup.

Method
- layout() : Iterasi children dari atas: y += child.preferredHeight + spacing, set boundingBox masing-masing

Hubungan
- Inheritence dengan Kelas Flexbox
- Komposisi dengan Kelas Popup dan Scene

2.10. Kelas HorizontalFlexbox

Implementasi Flexbox yang menyusun children dari kiri ke kanan. Digunakan untuk baris tombol, header dengan beberapa elemen sejajar, atau panel informasi pemain yang ditampilkan berdampingan.

Method
- layout()Iterasi children dari kiri: x += child.preferredWidth + spacing, set boundingBox masing-masing

Hubungan

- Inheritence dengan Kelas Flexbox

2.11. Kelas Popup

Dialog overlay yang muncul di atas Scene utama. Digunakan untuk konfirmasi aksi (beli properti, lelang, bangkrut), tampilan akta kepemilikan, dan panel likuidasi. Dikelola Scene melalui showPopup() dan collapsePopup().

Atribut

- container : VerticalFlexbox yang menampung semua elemen dalam popup
- isActive : True jika popup sedang ditampilkan di layar

Method

- draw() : Jika isActive, render container di atas Scene, biasanya di tengah layar
- update() : Jika isActive, delegasikan update ke container, terima input untuk elemen dalam popup
- add(element) : Tambah UIElement ke dalam container popup

Hubungan

- Komposisi dengan Kelas VerticalFlexbox

2.12. Kelas Scene

Merepresentasikan satu layar penuh aplikasi. Menyimpan root layout dan kumpulan popup. Memiliki akses ke ketiga manajer untuk mengambil data dan memicu aksi game dari interaksi UI.

Atribut

- root : VerticalFlexbox utama yang menampung seluruh UI scene
- popups : Map nama ke Popup, ditampilkan di atas root saat diperlukan
- currentPopup : Pointer ke popup aktif, null jika tidak ada
- gameManager, sceneManager, accountManager : Akses ke manajer untuk data dan trigger aksi

Method

- draw() : Draw root, lalu draw currentPopup di atasnya jika ada
- update() : Update root dan currentPopup
- showPopup(key) : Set currentPopup dari map berdasarkan key

- collapsePopup() : Set currentPopup ke null

Hubungan
- Inheritence dengan Kelas MainMenuScene, InGameScene
- Komposisi dengan Kelas Nimonspoli dan SceneManager
- Komposisi dengan Kelas VerticalFlexbox, Popup
- Agregasi dengan Kelas SceneManager

2.13. Kelas MainMenuScene

Scene untuk halaman utama sebelum permainan dimulai. Menampilkan opsi New Game dan Load Game, form input username pemain, pilihan jumlah pemain, dan leaderboard Account. Berinteraksi dengan AccountManager untuk login dan GameManager untuk memulai/memuat game.

Method
- (konstruktor/setup) : Bangun root layout: tombol new game, load game, form username, daftar akun, leaderboard

Hubungan
- Inheritence dengan Kelas Scene

2.14. Kelas InGameScene

Scene utama saat permainan berlangsung. Menampilkan papan permainan, informasi giliran, daftar perintah yang tersedia, dan status semua pemain. Menerima input perintah dari user (LEMPAR_DADU, BELI, GADAI, dll.) dan mendelegasikan ke Game melalui GameManager.

Method
- (konstruktor/setup)Bangun root layout: area papan, panel info pemain, input perintah, log transaksi

Hubungan
- Inheritence dengan Kelas Scene

2.15. Kelas SceneManager

Mengelola perpindahan antar scene dan mendelegasikan draw/update ke scene aktif. Titik koordinasi antara UI Layer dan Game Logic. Kelas ini memiliki referensi ke GameManager dan AccountManager untuk dibagikan ke Scene baru.

Atribut

- currentScene : Scene yang sedang aktif dan di-render
- accountManager, gameManager : Dibagikan ke Scene saat setScene() dipanggil

Method
- setScene(newScene) : Ganti currentScene, inject referensi manajer ke scene baru
- update() : Delegasikan ke currentScene.update() tiap frame
- draw() : Delegasikan ke currentScene.draw() tiap frame

Hubungan
- Komposisi dengan Kelas Nimonspoli
- Komposisi dengan Kelas Scene
- Agregasi dengan kelas Scene
- Agregasi dengan Kelas GameManager, AccountManager

2.16. Kelas GameException

Kelas dasar (base class) untuk seluruh exception dalam sistem permainan. Menyediakan struktur umum untuk menyimpan dan mengakses pesan error yang dapat digunakan oleh semua turunan exception.

Atribut
- message : Pesan error yang menjelaskan penyebab exception

Method
- GameException(message) : Konstruktor untuk menginisialisasi pesan error
- getMessage() : Mengembalikan pesan error

Hubungan
- Inheritance dari Kelas HandOverFlowException, BankruptException, InsufficientFundException, InvalidActionException, dan FileIOException
- Depedensi dengan Kelas Game dan GameManager

2.17. Kelas HandOverFlowException

Exception yang digunakan ketika jumlah kartu di tangan pemain melebihi batas maksimum yang diperbolehkan.

Method

- HandOverFlowException(message) : Konstruktor untuk menginisialisasi pesan error

Hubungan

- Inheritance dengan Kelas GameException
- Depedencies dengan Kelas Game

2.18. Kelas BankruptException

Exception yang digunakan ketika pemain tidak mampu membayar kewajiban dan dinyatakan bangkrut.

Atribut

- bankruptCode : Kode yang merepresentasikan jenis kebangkrutan (misalnya ke bank atau ke pemain lain)

Method

- BankruptException(code, message) : Konstruktor untuk menginisialisasi kode dan pesan error

Hubungan

- Inheritance dengan Kelas GameException
- Depedensi dengan Kelas Game

2.19. Kelas InsufficientFundException

Exception yang digunakan ketika pemain tidak memiliki dana yang cukup untuk melakukan suatu aksi finansial.

Atribut

- requiredAmount : Jumlah uang yang dibutuhkan
- currentAmount : Jumlah uang yang dimiliki pemain saat ini

Method

- InsufficientFundsException(req, cur, message) : Konstruktor untuk menginisialisasi kebutuhan dan kondisi dana

- getDeficit() : Mengembalikan selisih kekurangan dana (requiredAmount - currentAmount)

Hubungan
- Inheritance dengan Kelas GameException
- Depedensi dengan Kelas Player dan AuctionManager

2.20. Kelas InvalidActionException

Exception yang digunakan ketika pemain mencoba melakukan aksi yang tidak valid berdasarkan aturan permainan.
Method
- InvalidActionException(message) : Konstruktor untuk menginisialisasi pesan error

Hubungan
- Inheritance dengan Kelas GameException
- Depedensi dengan Kelas PropertyTile, StreetTile, dan Player

2.21. Kelas FileIOException

Exception yang digunakan untuk menangani kesalahan yang terjadi saat proses input/output file, seperti file tidak ditemukan atau gagal dibaca.
Method
- FileIOException(message) : Konstruktor untuk menginisialisasi pesan error

Hubungan
- Inheritance dengan Kelas GameException
- Depedensi dengan Kelas Loader

2.22. Kelas Game

Inti logika permainan. Menyimpan seluruh state yang diperlukan (board, player, giliran, dadu) dan mengatur alur satu sesi permainan dari awal hingga akhir. Setiap mekanisme besar (landing, giliran, kemenangan) dikoordinasikan oleh kelas ini melalui manager-manager yang dimilikinya.
Atribut
- board : Papan permainan berisi 40 Tile yang diinjakkan pemain
- players : Daftar 2–4 pemain aktif dalam satu sesi

- turn Manager : Mengatur urutan giliran, giliran ekstra, dan batas turn
- dice : Objek dadu untuk roll acak atau manual
- cardManager : Mengelola tiga tumpukan kartu (Kesempatan, Dana Umum, Kemampuan)
- auctionManager : Menjalankan mekanisme lelang properti
- bankruptcyManager : Menangani alur penanganan utang dan kebangkrutan pemain
- liquidationManager: Menangani proses likuidasi aset pemain
- logger : Mencatat semua kejadian signifikan selama permainan
- config : Data konfigurasi (harga, sewa, pajak) yang dibaca dari file
- gameOver : Flag penanda permainan sudah berakhir

Method
- run() : Loop utama permainan, memanggil startTurn() berulang sampai gameOver
- startTurn() : Dipanggil pada awal giliran untuk mereset flag player, beri kartu kemampuan, terima perintah
- endTurn() : Dipanggil pada akhir giliran untuk meng-update durasi efek, cek kondisi menang, pindah ke pemain berikutnya
- handleLanding(player) : Dispatcher utama untuk menentukan aksi berdasarkan jenis Tile yang diinjak
- checkWinCondition() : Cek apakah MAX_TURN tercapai atau hanya 1 pemain tersisa
- executeCommad(command: string) : Mengeksekusi perintah-perintah yang termasuk dalam aksi permainan.

Hubungan
- Komposisi dengan Kelas Board, Player, TurnManager, Dice, CardManager, AuctionManager, BankruptcyManager, Logger, Config, TradeManager, LiquidationManager
- Depedensi dengan Kelas HandOverFlowException, BankruptException, dan GameException
- Depedensi dengan Kelas BankruptcyManager, CardTile, FestivalTile, TaxTile, dan StreetTile

2.23. Kelas GameManager

Jembatan antara UI Layer dan Game Logic. Mengelola lifecycle permainan: membuat game baru, menyimpan, dan memuat. Menyembunyikan detail ConfigLoader dan Loader dari lapisan atas sehingga Scene hanya perlu memanggil GameManager.

Atribut
- currentGame : Pointer ke Game yang sedang berjalan, null jika belum ada

Method
- getCurrentGame() : Getter currentGame : diakses InGameScene untuk tampilkan state
- loadGame(folderName) : Delegasi ke Loader untuk restore state dari file save
- saveGame(folderName) : Delegasi ke SaveLoadManager untuk serialisasi state saat ini
- startNewGame() : Buat Game baru, load config, inisialisasi pemain dan board

Hubungan
- Komposisi dengan Kelas Nimonspoli
- Komposisi dengan Kelas Game
- Agregasi dengan Kelas SceneManager
- Dependensi dengan Kelas GameException dan Loader

2.24. Kelas TurnManager

Sumber kebenaran tunggal tentang 'giliran siapa sekarang'. Mengelola urutan, giliran ekstra saat double dadu, pengeluaran pemain bangkrut dari rotasi, dan pengecekan batas turn.

Atribut
- turnOrder : Urutan indeks pemain, dikocok acak di awal
- currentPlayerIndex : Posisi saat ini dalam turnOrder
- currentTurn : Nomor turn global untuk Logger dan checkWinCondition
- maxTurn : Dari config, jika <1 mode bankruptcy tanpa batas

Method
- getCurrentPlayerIndex() : Indeks pemain yang sedang giliran
- nextPlayer() : Maju ke pemain berikutnya, increment currentTurn
- grantExtraTurn() : Pemain yang sama giliran lagi setelah double dadu
- removePlayer(playerId) : Keluarkan pemain bangkrut dari turnOrder
- isMaxTurnReached() : Trigger kondisi menang MAX_TURN

Hubungan
- Komposisi dengan Kelas Game

2.25. Kelas Dice

Merepresentasikan dua dadu 6 sisi. Mendukung mode acak dan mode manual (ATUR_DADU). Menyediakan informasi yang dibutuhkan Game untuk pergerakan, sewa Utility, dan pendeteksian double.

Atribut

- die1, die2 : Nilai masing-masing dadu setelah roll terakhir
- manualMode : Jika True maka roll() gunakan nilai yang di-set manual

Method

- roll() : Mengubah nilai dadu secara random
- setManual(x,y) : Set nilai dadu manual, aktifkan manualMode
- getTotal() : die1+die2 untuk pergerakan dan sewa Utility
- isDouble() : die1==die2 untuk cek giliran ekstra dan penjara
- resetManualMode() : Kembali ke mode acak

Hubungan

- Komposisi dengan Kelas Game

2.26. Kelas Account

Akun pemain yang persisten antar sesi. Berbeda dari Player yang hanya hidup satu game, Account menyimpan identitas dan skor kumulatif untuk leaderboard. Operator overloading untuk pengurutan.

Atribut

- username, password : Kredensial login
- score : Skor kumulatif dari semua sesi

Method

- addScore(num) : Tambah skor setelah sesi selesai
- operator< / > / == : Pengurutan leaderboard berdasarkan score

Hubungan

- Agregasi dengan Kelas Player
- Komposisi dengan Kelas AccountManager

2.27. Kelas AccountManager

Titik akses tunggal untuk semua akun. Mengelola registrasi dan autentikasi. Dibagikan ke SceneManager dan Scene untuk proses login/register di MainMenuScene.

Atribut

- accounts : Map username ke Account untuk akses O(1)

Method

- addAccount(newAccount) : Daftarkan akun baru saat register
- getAccount(name,pass) : Return Account* jika kredensial cocok, null jika gagal

Hubungan

- Komposisi dengan Kelas Account
- Agregasi dengan Kelas Player

2.28. Kelas Player

Merepresentasikan satu pemain dalam sesi permainan. Menyimpan semua state personal: posisi, uang, properti, kartu tangan, dan efek aktif. Mengekspos operator overloading untuk perbandingan kekayaan dan manipulasi uang.

Atribut

- account : Referensi ke Account untuk username dan scoring pasca-game
- money : Uang tunai saat ini
- position : Indeks petak saat ini di Board
- status : ACTIVE / JAILED / BANKRUPT
- ownedProperties : Daftar PropertyTile yang dimiliki
- handCards : Max 3 AbilityCard di tangan
- jailTurnsAttempted : Counter percobaan keluar penjara (max 3)
- usedAbilityThisTurn : Pembatas 1 kartu kemampuan per giliran
- shieldActive : True jika ShieldCard aktif giliran ini

- discountPercent/Duration : Efek DiscountCard yaitu persen dan sisa durasi
- consecutiveDoubleCount : Counter double berturut di 3 langsung ke penjara

Method
- moveTo(position) : Set posisi pemain
- credit(amount) : Tambah uang, implementasi operator+=
- debit(amount) : Kurangi uang, lempar exception jika tidak cukup
- canAfford(amount) : Cek tanpa modifikasi sebelum debit
- addProperty/removeProperty(property) : Menambahkan atau menghapus properti
- addCard/dropCard() : Manipulasi handCards
- useAbilityCard(index,game) : Validasi canUse() lalu panggil card.use()
- enterJail/releaseFromJail() : Ubah status dan reset jailTurnsAttempted
- incrementJailAttempt() : Tambah counter percobaan keluar penjara
- resetTurnFlags() : Reset usedAbilityThisTurn, shieldActive, discountDuration tiap awal giliran
- getTotalWealth() : money + sum(landPrice semua properti) + sum(harga bangunan penuh)
- isBankrupt() : money <= 0 dan tidak ada aset yang bisa dilikuidasi
- toString() : String representasi untuk log dan UI
- operator< / > / == : Bandingkan getTotalWealth() untuk ranking pemenang

Hubungan
- Komposisi 2…4 dengan Kelas Game
- Komposisi dengan Kelas PropertyTile, PlayerStatus, dan AbilityCard
- Agregasi dengan Kelas Account
- Dependensi dengan Kelas InsufficientFundsException, InvalidActionException
- Depedensi dengan Kelas AuctionManager, BankruptcyManager, dan Tile

2.29. Kelas Board

Merepresentasikan papan permainan sebagai koleksi Tile. Bertanggung jawab atas kalkulasi pergerakan (termasuk deteksi melewati GO), pencarian tile terdekat, dan rendering visual papan ke layar.

Atribut

- tilesVector : Tile* urutan menentukan layout papan
- codeToIndex : Map kode huruf ke indeks untuk akses cepat

Method
- getTileByIndex(index) : Akses Tile berdasarkan posisi numerik
- getTileByCode(code) : Akses Tile berdasarkan kode huruf
- calculateNewPosition(pos,step,&passedGo) : Mengembalikan posisi baru dan *pass-by-reference* parameter passedGo
- getNearestRailroad(fromPos) : Iterasi tiles dari posisi saat ini, cari RailroadTile terdekat ke depan
- printBoard(players,turnInfo) : Render papan visual ke layar dengan posisi semua pemain

Hubungan
- Komposisi dengan Kelas Tile (40 buah) dan Kelas Game

2.30. Kelas Tile

Abstract base untuk semua petak. Mendefinisikan kontrak onLand() yang wajib diimplementasi setiap subclass. Polimorfisme ini memungkinkan Board menyimpan semua jenis petak dalam satu vector<Tile*> tanpa tahu detail masing-masing.

Atribut
- id : Indeks petak di board (0–39)
- code : Kode huruf unik (misal 'JKT', 'GO', 'PPH')
- name : Nama lengkap untuk ditampilkan ke user

Method
- onLand(player,game) : Pure virtual ini diimplementasi setiap subclass dengan perilaku berbeda
- getCode(), getName() : Getter dasar untuk Board dan Logger
- isOwnable() : Membedakan PropertyTile dari petak non-ownable

Hubungan
- Inheritence dengan Kelas PropertyTile, ActionTile, SpecialTile
- Komposisi dengan Kelas Board
- Depedensi dengan Kelas Player

2.31. Kelas PropertyTile

Abstract class untuk petak yang bisa dimiliki. Mengelola state kepemilikan, siklus gadai-tebus, dan efek festival. Mendefinisikan kontrak calculateRent() yang diimplementasi berbeda oleh setiap subclass.

Atribut

- owner : Referensi ke Player pemilik, null jika BANK
- status : BANK / OWNED / MORTGAGED
- mortgageValue : Dana yang diterima saat menggadaikan
- festivalMultiplier : Pengali sewa yaitu 1 (nonaktif), 2, 4, atau 8
- festivalDuration : Sisa giliran efek festival (0–3)

Method

- onLand(player, game) : Menentukan aksi yang terjadi ketika player mendarat di petak properti
- calculateRent(visitor,game) : Abstract rumus sewa berbeda per subclass
- SetOwner(owner) : Mengatur pemilik properti pada petak tersebut
- getOwner(owner) : Mendapatkan owner pemilik properti pada petak tersebut
- mortgage() : Status menjadi MORTGAGED, kembalikan mortgageValue ke owner
- redeem() : Status menjadi OWNED, bayar landPrice ke bank
- isOwned() : Mengembalikan status boolean apakah properti sudah dimiliki atau belum
- isMortgaged() : Mengembalikan status boolean apakah properti digadaikan
- applyFestival() : Gandakan festivalMultiplier (max 8x), reset durasi ke 3
- decrementFestivalDuration() : Kurangi durasi tiap akhir giliran pemilik
- resetFestivalIfExpired() : Reset multiplier ke 1 jika durasi 0
- toString() : Ringkasan properti untuk CETAK_PROPERTI dan CETAK_AKTA

Hubungan

- Inheritance dengan kelas Tile
- Inheritence dengan kelas StreetTile, RailroadTile, UtilityTile
- Asosiasi dengan Kelas Player
- Komposisi dengan Kelas PropertyStatus

- Depedensi dengan Kelas InvalidActionException
- Depedensi dengan Kelas AuctionManager dan BankruptcyManager

2.32. Kelas StreetTile

Properti paling kompleks. Mendukung pembangunan rumah/hotel, monopoli colorgroup, dan sewa bertingkat. Semua aturan detail (pemerataan bangunan, syarat monopoli, upgrade hotel) dienkapsulasi di sini.

Atribut

- colorGroup : Grup warna untuk deteksi monopoli
- landPrice : Harga beli juga untuk harga tebus dan getTotalWealth
- houseBuildCost, hotelBuildCost : Biaya bangun per level
- rentTable : 6 nilai sewa yaitu [tanah kosong, 1 rumah, ..., hotel]
- houseCount : Jumlah rumah saat ini (0–4)
- hasHotel : True jika sudah hotel (level maksimal)

Method

- calculateRent(visitor,game) : Lookup rentTable, kalikan festivalMultiplier, 2x jika monopoli tanpa bangunan
- hasMonopoly(game) : Cek apakah owner memiliki SEMUA Street dalam colorGroup yang sama
- canBuildHouse(game) : Cek monopoli dan pemerataan: houseCount tidak boleh > min colorgroup
- canBuildHotel(game) : Cek apakah semua petak colorgroup sudah 4 rumah
- buildHouse(), buildHotel() : Increment houseCount atau set hasHotel
- sellBuildingsToBank() : Jual semua bangunan 50% harga, reset ke tanah kosong
- getBuildingLevel() : Return level saat ini: 0–4 rumah atau 5 hotel
- canBeMortgaged(game) : False jika ada bangunan di manapun dalam colorgroup yang sama

Hubungan

- Inheritence dengan Kelas PropertyTile
- Depedensi dengan Kelas InvalidActionException dan Game

2.33. Kelas RailroadTile

Properti stasiun yang diperoleh otomatis saat pertama diinjak tanpa beli. Sewa bergantung jumlah railroad yang dimiliki owner, semakin banyak semakin mahal. Tidak bisa dibangun.

Method

- calculateRent(visitor,game) : Hitung jumlah RailroadTile milik owner, lookup railroadRentTable dari Config
- acquireAutomatically(player) : Set owner dan status OWNED tanpa proses beli atau lelang

Hubungan

- Inheritence dengan Kelas PropertyTile

2.34. Kelas UtilityTile

Properti utilitas (PLN dan PAM) yang diperoleh otomatis. Sewa dihitung dari total dadu yang dilempar visitor dikali faktor pengali sesuai jumlah utility yang dimiliki owner.

Method

- calculateRent(visitor,game) : Ambil dice.getTotal() dari Game, kalikan Config.getUtilityMultiplier(jumlah utility owner)
- acquireAutomatically(player) : Set owner dan status OWNED tanpa beli atau lelang

Hubungan

- Inheritence dengan Kelas PropertyTile

2.35. Kelas ActionTile

Abstract base untuk petak aksi yang tidak bisa dimiliki: TaxTile, FestivalTile, CardTile. Hanya mendefinisikan kontrak onLand() dimana setiap subclass mengimplementasi aksi yang berbeda.

Method

- onLand(player,game)Abstract yang dipanggil oleh Game.handleLanding()

Hubungan

- Inheritance dengan Kelas Tile
- Inheritence dengan Kelas TaxTile, FestivalTile, CardTile

2.36. Kelas TaxTile

Menangani PPH (pilihan flat atau persen kekayaan) dan PBM (flat langsung). Jika tidak mampu bayar maka bangkrut.

Atribut

- taxType : PPH atau PBM untuk menentukan alur di onLand()

Method

- onLand(player,game) : Dispatch ke processPPH atau processPBM
- calculatePPH(player) : 10% dari getTotalWealth() — dipanggil SETELAH pilihan dibuat
- processPPH(player,choice) : choice=1: bayar flat; choice=2: hitung lalu bayar persen
- processPBM(player) : Potong flat PBM langsung dari uang pemain

Hubungan

- Inheritence dengan Kelas ActionTile
- Komposisi dengan Kelas TaxType
- Depedensi dengan Kelas Game

2.37. Kelas FestivalTile

Saat pemain mendarat, ia memilih satu properti miliknya untuk dikenai efek festival yaitu sewa berlipat ganda selama 3 giliran pemilik. Mendelegasikan perubahan state ke PropertyTile.applyFestival().

Method

- onLand(player,game) : Tampilkan daftar properti milik player, minta pilihan, panggil activateFestivalOn
- activateFestivalOn(property) : Panggil property.applyFestival() karena PropertyTile yang kelola multiplier dan durasi

Hubungan

- Inheritence dengan Kelas ActionTile
- Depedensi dengan Kelas Game

2.38. Kelas CardTile

Petak yang memicu pengambilan kartu Kesempatan atau Dana Umum. Mendelegasikan draw ke CardManager, lalu kartu itu sendiri mengeksekusi efeknya via apply().

Atribut
- deckType : CHANCE atau COMMUNITY_CHEST untuk menentukan tumpukan mana yang diakses

Method
- onLand(player,game) : Tampilkan pesan, panggil drawAndApplyCard
- drawAndApplyCard(player,game) : Minta CardManager draw kartu sesuai deckType, panggil card.apply()

Hubungan
- Inheritence dengan Kelas ActionTile
- Komposisi dengan Kelas CardDeckType
- Depedensi dengan Kelas Game

2.39. Kelas SpecialTile

Menangani empat petak pojok papan: GO (gaji), Penjara (mampir vs tahanan), Bebas Parkir (tidak ada aksi), dan Pergi ke Penjara (pindah paksa dan giliran berakhir). Dispatch internal berdasarkan specialType.

Atribut
- specialType : GO / JAIL / FREE_PARKING / GO_TO_JAIL

Method
- onLand(player,game) : Switch ke handler sesuai specialType
- handleGo(player,game) : Panggil game.awardGoSalary() dan log
- handleJail(player,game) : ACTIVE: hanya mampir. JAILED: tampilkan opsi keluar penjara
- handleFreeParking() : Tidak ada aksi
- handleGoToJail(player,game) : Pindahkan ke Penjara, set JAILED, akhiri giliran tanpa GO salary

Hubungan
- Inheritence dengan Kelas Tile
- Komposisi dengan Kelas SpecialType

2.40. Kelas DrawCard

Abstract base untuk Kartu Kesempatan dan Dana Umum. Menyimpan efek sebagai function<void()> sehingga efek tiap kartu bisa dikonfigurasi fleksibel. Setelah apply(), kartu dikembalikan dan dikocok ulang ke tumpukan.

Atribut

- description : Teks yang ditampilkan saat kartu diambil
- effect : Lambda yang dieksekusi saat apply(), berisi logika efek spesifik

Method

- apply(player,game) : Eksekusi effect(), log ke Logger
- getDescription() : Tampilkan teks kartu sebelum efek dijalankan

Hubungan

- Inheritence dengan kelas ChanceCard, CommunityChestCard
- Komposisi dengan DrawCardDeck

2.41. Kelas ChanceCard

Implementasi konkret DrawCard untuk tiga Kartu Kesempatan: pergi ke stasiun terdekat, mundur 3 petak, dan masuk penjara. Efek dikonfigurasi saat konstruksi via lambda di DrawCard.effect.

Method

- apply(player,game) : Override apply() dengan efek kartu kesempatan dan manfaatkan Board.getNearestRailroad() atau player.enterJail()

Hubungan

- Inheritence dengan Kelas DrawCard

2.42. Kelas CommunityChestCard

Implementasi konkret DrawCard untuk tiga Kartu Dana Umum: terima M100 dari setiap pemain, bayar M700 dokter, bayar M200 ke setiap pemain. Melibatkan transfer uang antar pemain yang dikoordinasikan Game.

Method

- apply(player,game) : Override apply() untuk efek yang melibatkan semua pemain, akses game.players

Hubungan
- Inheritence dengan Kelas DrawCard

2.43. Kelas AbilityCard

Abstract base untuk Kartu Kemampuan Spesial yang disimpan di tangan pemain (max 3). Berbeda dari DrawCard yang langsung dieksekusi, AbilityCard disimpan dan dipilih kapan digunakan. canUse() memvalidasi konteks sebelum use().

Atribut
- name : Nama kartu ditampilkan di daftar tangan

Method
- canUse(player,game) : Validasi kapan bisa pakai kartu giliran ini
- use(player,game) : Abstract efek spesifik tiap subclass
- getName() : Mendapatkan nama jenis kartu
- serialize() : String untuk format save file yaitu 'JENIS NILAI DURASI'

Hubungan
- Inheritence dengan Kelas MoveCard, DiscountCard, ShieldCard, TeleportCard, LassoCard, DemolitionCard
- Komposisi dengan Kelas AbilityCardDeck
- Komposisi 0…3 dengan Kelas Player

2.44. Kelas MoveCard

Memindahkan pemain maju sejumlah petak. Nilai steps diroll SAAT kartu pertama kali didapat oleh CardManager bukan saat dipakai agar pemain bisa membuat keputusan berdasarkan nilai yang sudah diketahui.

Atribut
- steps : Jumlah petak yang diroll acak saat CardManager membuat kartu ini

Method
- use(player,game) : Hitung posisi baru, panggil player.moveTo(), trigger game.handleLanding()

Hubungan

- Inheritence dengan Kelas AbilityCard

2.45. Kelas DiscountCard

Memberikan diskon persentase acak untuk pembayaran sewa selama 1 giliran. Persen ditentukan saat kartu didapat. Efeknya disimpan di Player.discountPercent dan dikurangi di resetTurnFlags().

Atribut

- percent : Persentase diskon acak yang diset saat CardManager membuat kartu
- duration : Selalu 1

Method

- use(player,game) : Set player.discountPercent dan player.discountDuration = 1

Hubungan

- Inheritence dengan Kelas AbilityCard

2.46. Kelas ShieldCard

Memberikan imunitas terhadap semua tagihan dan sanksi selama 1 giliran. Setiap titik pembayaran (sewa, pajak, kartu Dana Umum) harus memeriksa player.shieldActive sebelum mengurangi uang.

Method

- use(player,game) : Set player.shieldActive = true, direset resetTurnFlags() awal giliran berikutnya

Hubungan

- Inheritence dengan Kelas AbilityCard

2.47. Kelas TeleportCard

Memindahkan pemain ke petak manapun di papan. Setelah pindah, onLand() petak tujuan tetap dieksekusi termasuk beli properti, bayar sewa, dll.

Method

- use(player,game) : Tampilkan semua kode petak, minta pilihan, panggil moveTo() lalu handleLanding()

Hubungan
- Inheritence dengan Kelas AbilityCard

2.48. Kelas LassoCard

Menarik satu pemain lawan yang berada DI DEPAN posisi pemain saat ini ke posisi pemain pemakai kartu. Pemain yang ditarik tidak mendapat efek landing.

Method
- use(player,game) : Cari pemain aktif dengan posisi > posisi player saat ini, pindahkan ke posisi player

Hubungan
- Inheritence dengan Kelas AbilityCard

2.49. Kelas DemolitionCard

Menghancurkan semua bangunan di satu properti milik pemain lawan. Pemilik tidak mendapat kompensasi, bangunan langsung dihapus dan properti kembali ke tanah kosong.

Method
- use(player,game) : Tampilkan properti lawan yang punya bangunan, pilih satu, reset bangunan tanpa bayar owner

Hubungan
- Inheritence dengan Kelas AbilityCard

2.50. Kelas "DrawCardDeck<T>" as DrawCardDeck

Generic class untuk mengelola pengambilan kartu berbasis random tanpa mekanisme discard. Digunakan untuk kartu yang tidak memerlukan siklus kembali (seperti Chance dan Community Chest), dengan pemilihan elemen secara acak menggunakan generator angka acak.

Atribut
- pile : Kumpulan kartu yang tersedia untuk diambil
- shuffler : Generator angka acak (mt19937) untuk memastikan distribusi acak yang baik
- picker : Distribusi uniform untuk memilih indeks kartu secara acak

Method

- draw() : Mengambil satu kartu secara acak dari pile menggunakan picker dan shuffler

Hubungan
- Komposisi dengan Kelas CardManager dan DrawCard

2.51. Kelas AbilityCardDeck

Class khusus untuk mengelola kartu ability dengan mekanisme siklus penuh (draw dan discard). Berbeda dengan DrawCardDeck, class ini mempertahankan state kartu yang sudah digunakan agar dapat digunakan kembali di masa depan.

Atribut
- drawPile : Kumpulan kartu ability yang siap diambil
- discardedPile : Kumpulan kartu ability yang sudah digunakan

Method
- draw() : Mengambil satu kartu dari drawPile
- discard(card) : Memindahkan kartu yang telah digunakan ke discardedPile

Hubungan
- Komposisi dengan kelas AbilityCard dan CardManager

2.52. Kelas CardManager

Pintu masuk tunggal untuk semua operasi kartu. Mengelola tiga tumpukan sekaligus, memastikan overflow tangan ditangani, dan kartu DrawCard dikembalikan setelah dipakai.

Atribut
- chanceDeckCard : Deck memiliki tiga kartu Kesempatan
- communityDeckCard : Deck memiliki tiga kartu Dana Umum
- abilityDeck : Tumpukan 15 AbilityCard (4+3+2+2+2+2)

Method
- drawChanceCard/drawCommunityCard() : Draw dari tumpukan, shuffle jika habis
- drawAbilityCard() : Draw satu AbilityCard, inisialisasi nilai acak (steps MoveCard, dll.)

- giveTurnStartAbility(player) : Dipanggil Game.startTurn() untuk SEMUA player aktif dimana tiap orang dapat 1 kartu
- handleHandOverflow(player) : Jika handCards > 3, tampilkan pilihan drop kemudian lempar HandOverFlowException
- discardAbilityCard(card) : Setelah dipakai, buang ke discardPile abilityDeck
- returnChanceCard(card) : Setelah apply(), kembalikan ke tumpukan dan reshuffle

Hubungan
- Komposisi dengan Kelas Game
- Komposisi dengan Kelas DrawCardDeck dan AbilityCardDeck

2.53. Kelas AuctionManager

Menjalankan mekanisme lelang secara urut. Mulai dari pemain BERIKUTNYA setelah pemicu, loop sampai (jumlah_pemain-1) pass berturut-turut. Menangani edge case semua pemain pass tanpa ada yang bid.

Method
- runAuction(property,bidders,trigger,game) : Tentukan urutan dari turnOrder, loop bid/pass sampai kondisi selesai
- validateBid(player,amount) : amount > bid sebelumnya DAN player.canAfford(amount)
- determineWinner() : Return pemain dengan bid tertinggi
- finalizeAuction() : Transfer uang dan ubah status properti ke OWNED

Hubungan
- Komposisi dengan Kelas Game
- Depedensi dengan Kelas Player, PropertyTile, Game, dan InsufficientFundsException
- Depedensi dengan Kelas BankruptcyManager

2.54. Kelas TradeManager

Bertanggung jawab untuk mengelola seluruh proses pertukaran (trade) antar pemain, mulai dari pencatatan penawaran trade, negosiasi, hingga eksekusi transaksi properti antar pemain.

Atribut
- tradesToPlayer : Kumpulan objek trade yang sedang aktif atau menunggu persetujuan

Method

- getTrades() : Mengembalikan seluruh daftar trade yang tersedia
- addTrade(giver, getter, propGive, propGet) : Membuat dan menambahkan penawaran trade baru antara dua pemain
- executeTrade(tradeId) : Mengeksekusi trade berdasarkan ID, termasuk pertukaran properti antar pemain
- negotiateTrade() : Menangani proses negosiasi trade (misalnya menerima, menolak, atau modifikasi penawaran)

Hubungan
- Komposisi dengan Kelas Game
- Agregasi dengan Kelas TradeToPlayer

2.55. Kelas TradeToPlayer

Merepresentasikan satu entitas transaksi trade antara dua pemain, termasuk detail properti yang ditukar serta identitas pihak yang terlibat.

Atribut
- staticNextID : Counter statis untuk menghasilkan ID unik setiap trade
- tradeID : Identitas unik dari trade
- propGive : Daftar properti yang ditawarkan oleh pemberi
- propGet : Daftar properti yang diminta dari penerima
- giver : Pemain yang menawarkan trade
- getter : Pemain yang menerima tawaran trade

Method
- getTradeID() : Mengembalikan ID trade
- getPropGive() : Mengembalikan daftar properti yang ditawarkan
- getPropGet() : Mengembalikan daftar properti yang diminta

Hubungan
- Agregasi dengan Kelas TradeManager

2.56. Kelas LiquidationManager

Bertanggung jawab untuk menangani proses likuidasi aset pemain, terutama ketika pemain harus menjual atau melepas properti untuk memenuhi kewajiban finansial.

Method
- _calculatePrice(player, property) : Menghitung nilai jual suatu properti milik pemain
- _liquidateAssets(player, targetProp) : Melakukan proses likuidasi terhadap daftar properti yang ditentukan untuk memenuhi kebutuhan dana

Hubungan
- Komposisi dengan Kelas Game
- Depedensi dengan Kelas InvalidActionExceptioon, Player, dan PropertyTile

2.57. Kelas BankruptcyManager

Menangani seluruh alur kebangkrutan: estimasi kemampuan lunas, panel likuidasi interaktif, deklarasi bangkrut, dan pengalihan aset. Membedakan bangkrut ke Bank (properti dilelang) vs ke pemain lain (aset langsung dialihkan).

Method
- canRecoverDebt(player,amount) : Bandingkan estimateMaxLiquidationValue() dengan amount
- estimateMaxLiquidationValue(player) : Hitung max dana: jual properti OWNED + gadai OWNED. Properti MORTGAGED = 0 net
- resolveDebt(debtor,amount,game) : Mengoordinasikan seluruh alur penyelesaian utang dari awal hingga akhir
- liquidateAssets(player,targetAmount) : Tampilkan panel interaktif jual/gadai sampai kewajiban terpenuhi
- declareBankruptToBank(player,game) : Uang sisa ke Bank, properti ke BANK, lelang satu per satu
- declareBankruptToPlayer(player,creditor,game) : Transfer semua aset termasuk yang tergadai langsung ke creditor
- transferAssets(from,to) : Pindahkan semua ownedProperties dan uang sisa
- auctionBankReturnedProperties(game) : Loop setiap properti yang ke BANK, panggil AuctionManager.runAuction()

Hubungan
- Komposisi dengan Kelas Game
- Depedensi dengan Kelas AuctionManager, Player, PropertyTile, LiquidationManager dan Game

2.58. Kelas Logger

Mencatat semua kejadian signifikan secara otomatis selama permainan. Berjalan di latar belakang tanpa perintah khusus. Entries disimpan di memori dan diserialisasi bersama state game saat SIMPAN.

Atribut
- entries : Vector semua LogEntry yang terakumulasi selama sesi

Method
- log(turn,username,actionType,detail) : Buat LogEntry baru
- printLog() : Tampilkan semua atau N entri terakhir (CETAK_LOG dengan/tanpa argumen)
- getEntries() : Mengembalikan seluruh log yang tersimpan
- clear() : Menghapus seluruh isi log atau reset histori
- serialize() : Ubah semua entries ke format string untuk disimpan di file save

Hubungan
- Komposisi dengan Kelas LogEntry dan Kelas Game

2.59. Kelas LogEntry

Value object yang merepresentasikan satu kejadian dalam log. Tidak memiliki behavior kompleks hanya menyimpan data dan menghasilkan string representasinya.

Atribut
- turn : Nomor giliran saat kejadian terjadi
- username : Pemain yang terlibat dalam aksi
- actionType : Jenis aksi: DADU, BELI, SEWA, PAJAK, KARTU, FESTIVAL, BANGKRUT, dll.
- detail : Deskripsi detail: nilai uang, nama properti, hasil dadu

Method
- toString() : Format: '[Turn N] Username | JENIS | Detail'

Hubungan
- Komposisi dengan Kelas Logger

2.60. Kelas Config

Value object yang menyimpan seluruh data konfigurasi hasil baca dari file. Menjadi sumber kebenaran tunggal untuk semua nilai yang bisa dikustomisasi (harga, sewa, pajak). Diakses oleh komponen Game Logic via Game.

Atribut

- propertyConfigs : Data setiap properti dari property.txt
- railroadRentTable : Tabel sewa railroad dari railroad.txt
- utilityMultiplierTable : Tabel pengali utility dari utility.txt
- taxConfig : PPH_FLAT, PPH_PERSENTASE, PBM_FLAT dari tax.txt
- specialConfig : GO_SALARY, JAIL_FINE dari special.txt
- miscConfig : MAX_TURN, SALDO_AWAL dari misc.txt

Method
- getPropertyConfig(code) : Return config properti berdasarkan kode petak
- getRailroadRent(count) : Lookup biaya sewa berdasarkan jumlah railroad yang dimiliki
- getUtilityMultiplier(count) : Lookup faktor pengali berdasarkan jumlah utility yang dimiliki

Hubungan
- Komposisi dengan Kelas Game

2.61. Kelas Loader

Bertanggung jawab atas deserialisasi file save. Membaca file berbasis teks dan mengonversinya menjadi struktur objek yang digunakan oleh sistem seperti Player, PropertyTile, CardManager, LogEntry, serta berbagai konfigurasi permainan.

Method
- getIfstream(path) : Membuka file input stream dari path yang diberikan dengan validasi error (digunakan sebagai helper internal)
- loadPlayers(path) : Membaca data pemain dari file dan merekonstruksi seluruh objek Player beserta atributnya
- loadProperties(path) : Membaca data properti dan membentuk objek PropertyTile sesuai state tersimpan
- loadDecks(path) : Membaca data deck dan merekonstruksi CardManager beserta isi kartu
- loadLogs(path) : Membaca histori permainan dan mengembalikannya sebagai kumpulan LogEntry
- loadPropertiesConfig(path) : Membaca konfigurasi properti (harga, warna, rent table, dll)
- loadRailroadsConfig(path) : Membaca tabel sewa untuk railroad
- loadUtilitiesConfig(path) : Membaca multiplier untuk utility
- loadTaxConfig(path) : Membaca konfigurasi pajak (PPH dan PBM)
- loadSpecialConfig(path) : Membaca konfigurasi tile spesial (GO, JAIL, dll)

- loadMiscConfig(path) : Membaca konfigurasi tambahan permainan
- loadAccounts(path) : Membaca data akun dan merekonstruksi objek Account

Hubungan
- Depedensi dengan Kelas FileIOException
- Depedensi dengan Kelas GameManager, dan AccountManager

3. Justifikasi

Desain diagram kelas pada sistem Nimonspoli dibuat dengan pendekatan OOP yang berfokus pada pembagian tanggung jawab yang jelas, agar setiap bagian sistem mudah dipahami dan dikelola. Secara umum, sistem dibagi menjadi beberapa bagian utama seperti tampilan (UI), logika permainan, serta pengelolaan data dan penyimpanan. Misalnya, kelas Game menangani jalannya permainan, sementara tampilan diatur oleh UIElement dan Scene. Selain itu, terdapat kelas seperti GameManager, SceneManager, dan AccountManager yang membantu mengatur alur program agar lebih terstruktur. Dari sisi desain, setiap kelas memiliki peran yang spesifik sehingga tidak saling tumpang tindih. Prinsip *encapsulation* diterapkan dengan baik melalui pemisahan atribut privat dan method publik pada hampir seluruh kelas. Konsep *inheritance* digunakan untuk mengelompokkan kelas yang memiliki kesamaan, seperti pada Tile, PropertyTile, dan berbagai jenis kartu. Hal ini memudahkan pengembangan karena perilaku umum bisa ditulis sekali di kelas induk, lalu disesuaikan di kelas turunan. Selain itu, banyak kelas yang saling terhubung sebagai bagian dari sistem utama, seperti Game yang memiliki Board, Player, dan berbagai manager lain. Untuk hubungan yang hanya digunakan sesekali, digunakan dependency agar tidak membuat hubungan yang terlalu kuat antar kelas.

Kelebihan dari desain ini adalah strukturnya yang cukup rapi dan mudah dikembangkan. Jika ingin menambahkan fitur baru, seperti jenis kartu atau tile baru, cukup menambahkan kelas baru tanpa harus mengubah banyak bagian lain. Pemisahan sistem kartu menjadi dua bagian yaitu DrawCardDeck dan AbilityCardDeck juga membuat desain lebih jelas sesuai kebutuhan permainan. Selain itu, adanya pembagian tugas ke berbagai manager seperti AuctionManager, TradeManager, dan BankruptcyManager membantu agar logika tidak menumpuk di satu kelas saja. Namun, desain ini juga memiliki beberapa kekurangan. Jumlah kelas yang cukup banyak membuat diagram terlihat kompleks dan membutuhkan waktu untuk dipahami secara keseluruhan. Beberapa kelas seperti Game masih memiliki banyak tanggung jawab karena terhubung dengan banyak komponen lain. Selain itu, penggunaan tipe data seperti tuple pada kelas Config kurang mudah dibaca dibandingkan jika menggunakan kelas khusus. Ada juga beberapa detail implementasi seperti penggunaan pointer yang perlu diperhatikan agar tidak menimbulkan masalah saat coding.

Selama proses perancangan, terdapat beberapa kendala. Salah satunya adalah menentukan pembagian tugas antar kelas agar tidak saling tumpang tindih, terutama antara Game, Player, dan berbagai manager. Selain itu, desain sistem kartu juga sempat diubah karena pendekatan awal kurang sesuai dengan kebutuhan permainan. Tantangan lain adalah menyesuaikan desain yang baik dengan implementasi nyata, khususnya dalam penggunaan bahasa C++. Secara keseluruhan, desain ini merupakan hasil penyempurnaan bertahap agar sistem menjadi lebih terstruktur, mudah dikembangkan, dan tetap sesuai dengan kebutuhan permainan sesuai pada spesifikasi.

4. Penerapan Konsep OOP

4.1. Inheritance & Polymorphism

Pada implementasi program Nimonspoli, konsep inheritance dan polymorphism digunakan untuk memodelkan objek-objek permainan yang memiliki perilaku umum namun tetap memiliki karakteristik khusus. Pendekatan ini diterapkan pada beberapa bagian utama sistem, seperti hierarki class Tile pada board, sistem kartu ability, serta komponen GUI. Dengan inheritance, atribut dan perilaku dasar dapat diwariskan dari class induk ke class turunan, sedangkan polymorphism memungkinkan objek-objek turunan diperlakukan sebagai objek bertipe induk, namun tetap menjalankan perilaku spesifik masing-masing saat runtime.

File Tile.hpp

```
class Tile {
protected:
    int id;
    std::string code;
    std::string name;

public:
    Tile(int id, std::string code, std::string name);
    virtual ~Tile();

    virtual void onLand(Player* player, Game* game) = 0;
```

```
    virtual bool isOwnable() const;};
```

Class PropertyTile kemudian mewarisi Tile dan menambahkan atribut serta operasi yang khusus untuk petak properti, seperti owner, status, calculateRent, dan getLandPrice.

File PropertyTile.hpp

```
class Tile {
protected:
    int id;
    std::string code;
    std::string name;

public:
    Tile(int id, std::string code, std::string name);
    virtual ~Tile();

    virtual void onLand(Player* player, Game* game) = 0;
    virtual bool isOwnable() const;
};
```

Selanjutnya, class seperti StreetTile, RailroadTile, dan UtilityTile merupakan turunan dari PropertyTile yang mengimplementasikan perhitungan sewa dengan cara berbeda sesuai aturan masing-masing.

File StreetTile.hpp, RailroadTile.hpp, dan UtilityTile.hpp

```
class StreetTile : public PropertyTile {
public:
    int calculateRent(Player* visitor, Game* game) override;
    int getLandPrice() const override;
```

```
};

class RailroadTile : public PropertyTile {
public:
    int calculateRent(Player* visitor, Game* game) override;
    int getLandPrice() const override;
};

class UtilityTile : public PropertyTile {
public:
    int calculateRent(Player* visitor, Game* game) override;
    int getLandPrice() const override;
};
```

Konsep polymorphism terlihat ketika game hanya berinteraksi dengan pointer bertipe Tile, tanpa perlu mengetahui jenis spesifik petak tersebut. Method yang dipanggil akan menyesuaikan dengan tipe objek sebenarnya saat runtime.

File Game.cpp

```
void Game::handleLanding(Player& player) {
    Tile* tile = board.getTileByIndex(player.getPosition());
    if (tile != nullptr) {
        tile->onLand(&player, this);
    }
}
```

Dengan menggunakan konsep OOP ini terkait inheritance & polymorphism, logika program lebih fleksibel karena tidak memerlukan percabangan berdasarkan tipe objek, serta mempermudah penambahan jenis tile baru di masa depan. Penggunaan inheritance dan polymorphism pada sistem ini sesuai karena semua properti memiliki kesamaan sebagai petak yang dapat dimiliki pemain, namun memiliki aturan bisnis yang berbeda. Dengan

desain ini, kode menjadi lebih modular, fleksibilitas, serta kemudahan pengembangan sistem, karena setiap komponen dapat diperluas tanpa harus mengubah struktur utama yang sudah ada..

4.2. Method/Operator Overloading

Konsep overloading digunakan untuk menyediakan beberapa versi operasi yang memiliki tujuan sama, tetapi menerima parameter atau konteks yang berbeda. Penerapan ini membuat antarmuka class menjadi lebih konsisten dan memudahkan pemanggilan method sesuai kebutuhan tanpa harus membuat nama fungsi yang terlalu banyak. Di proyek ini, bentuk overloading yang paling terlihat adalah method overloading dan operator overloading.

Untuk method overloading, salah satu contoh yang jelas terdapat pada class StreetTile. Class ini memiliki beberapa method dengan nama yang sama, tetapi parameter berbeda, misalnya hasMonopoly, canBuildHouse, canBuildHotel, dan canBeMortgaged.

File StreetTile.hpp

```
bool hasMonopoly(Game* game);
bool canBuildHouse(Game* game);
bool canBuildHotel(Game* game);

bool hasMonopoly(const std::vector<const StreetTile*>& group) const;
bool canBuildHouse(const std::vector<const StreetTile*>& group) const;
bool canBuildHotel(const std::vector<const StreetTile*>& group) const;

bool canBeMortgaged(Game* game) const override;
bool canBeMortgaged(const std::vector<const StreetTile*>& group) const;
```

Pada kasus ini, alasan penggunaan method overloading adalah karena operasi yang dilakukan secara konsep sama, misalnya memeriksa apakah properti bisa dibangun atau digadaikan, tetapi sumber informasi yang dipakai bisa berbeda. Versi dengan parameter Game cocok digunakan ketika object mengambil data langsung dari state permainan secara global, sedangkan versi dengan parameter vector<const StreetTile*> digunakan ketika kumpulan properti yang relevan sudah tersedia. Dengan demikian, method yang sama dapat digunakan secara fleksibel sesuai konteks pemanggilan.

Keuntungan pendekatan ini adalah antarmuka class menjadi lebih rapi karena operasi yang sejenis tetap menggunakan nama yang sama, kode lebih mudah dibaca karena nama method langsung merepresentasikan tujuan logikanya, serta implementasi menjadi lebih fleksibel tanpa perlu membuat fungsi baru yang redundan.

Contoh implementasi method overloading juga terlihat pada fungsi canBuildHouse berbasis group, yang melakukan validasi terhadap seluruh properti dalam satu kelompok.

File StreetTile.cpp

```
bool StreetTile::canBuildHouse(const std::vector<const StreetTile*>& group) const {
    if (!hasMonopoly(group) || isMortgaged() || hasHotel || houseCount >= 4) {
        return false;
    }

    int minHouseCount = houseCount;
    for (const StreetTile* street : group) {
        if (street == nullptr || street->hasHotelBuilt()) {
            return false;
        }
        minHouseCount = std::min(minHouseCount, street->getHouseCount());
    }

    return houseCount <= minHouseCount;
}
```

Untuk operator overloading, penerapannya terlihat pada class Player dan Account. Pada class Player, operator perbandingan di-overload untuk membandingkan total kekayaan pemain.

File Player.cpp

```
bool Player::operator<(const Player& other) const {
    return this->getTotalWealth() < other.getTotalWealth();
}
bool Player::operator>(const Player& other) const {
    return this->getTotalWealth() > other.getTotalWealth();
}
bool Player::operator==(const Player& other) const {
    return this->getTotalWealth() == other.getTotalWealth();
}
```

Selain itu, pada class Account, operator digunakan untuk membandingkan akun berdasarkan skor dan username.

File Account.cpp

```
bool operator<(const Account& other) const {
    if (this->score != other.score) return this->score < other.score;
    return this->username < other.username;
}
bool operator>(const Account& other) const {
    if (this->score != other.score) return this->score > other.score;
    return this->username > other.username;
}
bool operator==(const Account& other) const {
    return this->score == other.score;
}
```

Operator overloading digunakan karena perbandingan antar object Player dan Account merupakan operasi yang wajar dalam konteks domain aplikasi. Dalam permainan, membandingkan pemain berdasarkan total kekayaan penting untuk menentukan peringkat, sedangkan pada akun digunakan untuk keperluan leaderboard.

Keuntungan dari operator overloading adalah ekspresi perbandingan menjadi lebih natural, misalnya menggunakan bentuk playerA > playerB, logika pembandingan tersentralisasi dalam class sehingga tidak perlu ditulis ulang di berbagai tempat, serta kode menjadi lebih bersih dan lebih mudah dipahami. Penerapan overloading pada program ini memberikan fleksibilitas dalam penggunaan method dan operator, menjaga konsistensi antarmuka class, serta meningkatkan keterbacaan dan kemudahan pengembangan kode di masa depan.

4.3. Template & Generic Classes

Konsep template dan generic class digunakan untuk menangani berbagai jenis objek yang memiliki pola pengelolaan serupa, tetapi berbeda tipe data konkretnya. Penggunaan template bertujuan untuk menghindari duplikasi kode dengan menyediakan satu struktur atau fungsi yang dapat digunakan untuk berbagai tipe data selama memenuhi operasi yang dibutuhkan. Pendekatan ini diterapkan terutama pada pengelolaan deck kartu dan proses serialisasi data game.

Salah satu contoh utama terdapat pada class DrawCardDeck yang didefinisikan sebagai generic class dengan parameter tipe T. Class ini digunakan untuk merepresentasikan kumpulan kartu yang dapat diambil secara acak, tanpa bergantung pada jenis kartu tertentu.

File DrawCardDeck.hpp

```
template <class T>
class DrawCardDeck
{
private:
   std::vector<T*> pile;
   std::mt19937 shuffler;
   std::uniform_int_distribution<> picker;
public:
   DrawCardDeck() : pile({}) {}
```

```
    DrawCardDeck(std::vector<T*> pile) : pile(pile), shuffler(std::random_device{}()), picker(0, (int)pile.size() - 1){}
    T* draw() {
        return pile[picker(shuffler)];
    }
    const std::vector<T*>& getPile() const { return pile; }
};
```

Class ini memungkinkan pengelolaan berbagai jenis deck, seperti ChanceCard dan CommunityChestCard, tanpa perlu membuat implementasi terpisah untuk masing-masing jenis kartu. Secara konsep, semua kartu tersebut memiliki pola yang sama, yaitu disimpan dalam suatu kumpulan, dapat diambil secara acak, dan diakses melalui struktur data yang sama. Oleh karena itu, penggunaan template menjadi solusi yang tepat.

Keuntungan dari penggunaan generic class ini adalah mengurangi duplikasi kode karena satu class dapat digunakan untuk berbagai tipe kartu, mempermudah proses maintenance karena perubahan hanya perlu dilakukan di satu tempat, serta meningkatkan fleksibilitas dalam pengembangan karena penambahan jenis kartu baru tidak memerlukan perubahan pada struktur deck.

Penggunaan template ini juga terlihat pada implementasi kartu konkret seperti ChanceCard dan CommunityChestCard, yang memiliki perilaku berbeda tetapi tetap dapat dikelola oleh DrawCardDeck karena tidak bergantung pada tipe spesifik.

File ChanceCard.cpp dan CommunityChestCard.cpp

```
StepbackCard::StepbackCard() : ChanceCard(
    "Mundur 3 petak.",
    [](Player* p, Game* g) {
        Board& board = g->getBoard();
        bool dummy = false;
        int stepbackIndex = board.calculateNewPosition(p->getPosition(), -3, dummy);
        p->moveTo(stepbackIndex);
        board.getTileByIndex(stepbackIndex)->onLand(p, g);
    }
) {}
```

```
BirthdayCard::BirthdayCard() : CommunityChestCard(
    "Ini adalah hari ulang tahun Anda. Dapatkan M100 dari setiap pemain.",
    [](Player* p, Game* g) {
        for (auto player : g->getPlayers()) {
            if (player.getId() != p->getId()) {
                p->receive(100);
            }
        }
    }
) {}
```

Penggunaan template dan generic class pada program ini memberikan berbagai keuntungan, seperti meningkatkan reusability, mengurangi duplikasi kode, mempermudah maintenance, serta mendukung pengembangan fitur baru. Dengan pendekatan ini, program menjadi lebih fleksibel dan tetap menjaga type safety yang merupakan salah satu keunggulan bahasa C++.

4.4. Exception

Exception digunakan untuk menangani kondisi error atau kondisi permainan yang tidak valid secara terstruktur. Dibandingkan hanya mengembalikan nilai khusus seperti -1 atau false, exception membuat program dapat langsung memisahkan alur normal dan alur error. Hal ini penting karena dalam game terdapat banyak kejadian khusus, misalnya pemain tidak punya uang cukup, pemain benar-benar bangkrut, file save gagal dibuka, atau aksi yang dilakukan pemain tidak sesuai aturan.

Struktur exception pada proyek ini dimulai dari class dasar GameException, yang kemudian diturunkan menjadi beberapa exception yang lebih spesifik seperti BankruptException, InsufficientFundException, InvalidActionException, dan FileIOException.

File GameException.hpp

```
class GameException : public exception {
```

```
protected:
    string message;

public:
    explicit GameException(string message);
    virtual ~GameException() = default;

    string getMessage() const;
    virtual const char* what() const noexcept override;
};
```

File GameException.cpp

```
GameException::GameException(string message) : message(message) {}

const char* GameException::what() const noexcept {
    return message.c_str();
}
```

Penggunaan exception seperti ini sesuai karena semua error pada game tetap memiliki karakteristik umum, yaitu membawa pesan kesalahan. Dengan menyediakan satu induk bersama, seluruh exception turunannya menjadi lebih konsisten dan lebih mudah dikelola. Ini juga memberi keuntungan dari sisi desain OOP karena exception dapat dikelompokkan dalam satu hierarki yang jelas.

Contoh lainnya ada pada InsufficientFundException dan BankruptException. Keduanya digunakan pada method Player::pay().

File BankruptException.hpp

```
class BankruptException : public GameException {
private:
    int bankruptCode;
```

```
public:
    BankruptException(int code, string message);
    int getBankruptCode() const;
};
```

File InsufficientFundException.hpp

```
class InsufficientFundException : public GameException
{
private:
    int requiredAmount;
    int currentAmount;
public:
    InsufficientFundException(int requiredAmount, int currentAmount);
    int getDeficit() const;
};
```

File Player.cpp

```
void Player::pay(int amount) {
    if (amount > this->money) {
        if (this->ownedProperties.empty()) {
            throw BankruptException(1, "gagal bayar, ga punya properti juga");
        }
        throw InsufficientFundException(amount, this->money);
    }
    this->money -= amount;
}
```

Pada kasus ini, exception digunakan karena kegagalan pembayaran bukan sekadar hasil boolean sederhana. Ada dua kondisi berbeda, yaitu pemain tidak cukup uang tetapi masih punya aset, sehingga bisa masuk proses likuidasi; serta pemain tidak cukup uang dan juga tidak punya aset, sehingga dinyatakan bangkrut. Exception digunakan dikarenakan kondisi tersebut adalah kejadian khusus yang mengubah alur permainan secara signifikan. Jika hanya memakai nilai return biasa, caller harus terus-menerus memeriksa status tambahan secara manual. Dengan exception, logika error bisa langsung "naik" ke level yang memang bertanggung jawab menanganinya.

Keuntungannya dari penggunaan exception pada kode tersebut adalah pemisahan alur normal dan alur kegagalan menjadi lebih jelas, informasi error lebih kaya karena exception dapat membawa data tambahan seperti requiredAmount, currentAmount, atau bankruptCode, lebih mudah diperluas jika nanti ada jenis kegagalan pembayaran baru.

Penerapan exception pada Nimonspoli ini penting karena game memiliki banyak kondisi tidak normal yang membutuhkan penanganan berbeda-beda. Exception membantu menjaga kode tetap bersih, memisahkan logika normal dari logika error, dan membuat sistem lebih mudah dikembangkan serta dipelihara.

4.5. C++ Standard Template Library

4.5.1. Penggunaan STL Map

File BoardBuilder.cpp

```
#include <map>

void BoardBuilder::build(
    Board& board,
    const std::vector<std::tuple<int, std::string, std::string, std::string, std::string, int, int, std::vector<int>>>& propConfigs,
    const std::vector<std::tuple<int, std::string, std::string, std::string, std::string>>& actionConfigs,
    const std::map<int, int>& railroadRent,
    const std::map<int, int>& utilityMult
)
```

```
{
   struct TileInfo {
      int id;
      std::string code, name, type, color;
      int landPrice = 0, mortgageValue = 0;
      std::vector<int> rentTable;
      bool isProperty = false;
   };

   std::map<int, TileInfo> tileMap;
}
```

std::map cocok digunakan di sini karena program perlu menggabungkan dua sumber data yang berbeda, yaitu konfigurasi properti dan konfigurasi petak aksi. Keduanya punya ID posisi. Dengan map, data bisa disimpan berdasarkan ID yang sama dalam satu struktur.

4.5.2. Penggunaan STL Algorithm

File StreetTile.cpp

```
#include <algorithm>
bool StreetTile::canBuildHouse(const std::vector<const StreetTile*>& group) const {
   if (!hasMonopoly(group) || isMortgaged() || hasHotel || houseCount >= 4) {
      return false;
   }

   int minHouseCount = houseCount;
   for (const StreetTile* street : group) {
      if (street == nullptr || street->hasHotelBuilt()) {
         return false;
      }
      minHouseCount = std::min(minHouseCount, street->getHouseCount());
   }
```

```
    return houseCount <= minHouseCount;
}
```

Pada class StreetTile, STL Algorithm digunakan melalui fungsi std::min() dari header <algorithm>. Fungsi ini dipakai pada method canBuildHouse() untuk mencari jumlah rumah paling sedikit dalam satu color group. Informasi ini dibutuhkan karena aturan pembangunan rumah pada Nimonspoli harus merata. Pemain hanya boleh membangun rumah pada properti yang jumlah rumahnya masih minimum dibanding properti lain dalam group yang sama.

Penggunaan std::min() sesuai dalam kasus ini karena proses validasi pembangunan rumah membutuhkan operasi pencarian nilai minimum. Dengan menggunakan fungsi bawaan STL, kode menjadi lebih ringkas, mudah dibaca, dan mengurangi risiko kesalahan dibanding menulis logika perbandingan manual. Method ini juga membuat aturan pemerataan bangunan lebih jelas karena keputusan akhir dibuat berdasarkan nilai minimum rumah dalam color group.

4.5.3. Penggunaan STL vector

File BankruptcyManager.cpp

```
#include <vector>
void BankruptcyManager::transferAssets(Player& from, Player& to) {
    to.receive(from.getMoney());
    from.pay(from.getMoney());

    vector<PropertyTile*> propertiesToTransfer = from.getOwnedProperties();
    for (PropertyTile* prop : propertiesToTransfer) {
        prop->setOwner(&to);
        to.addProperty(prop);
        from.removeProperty(prop);
    }
}
```

std::vector cocok digunakan dalam BankruptcyManager karena program perlu mengelola daftar aset pemain yang jumlahnya bisa lebih dari satu. Dengan vector, proses iterasi, pemindahan aset, penyitaan properti, dan penghapusan properti menjadi lebih sederhana, terstruktur, dan mudah dibaca.

4.6. Konsep OOP lain

4.6.1. Abstract Base Class

Pada implementasi Nimponspoli ini beberapa class dibuat sebagai abstract base class, yaitu class yang tidak dimaksudkan untuk diinstansiasi langsung, melainkan menjadi kerangka bagi class turunannya. Berikut ini salah satu contoh yang terlihat untuk Tile

File Tile.hpp

```
class Tile {
public:
    virtual ~Tile();
    virtual void onLand(Player* player, Game* game) = 0;
    virtual bool isOwnable() const;
};
```

Konsep ini digunakan karena ada beberapa object yang secara konsep punya bentuk umum, tetapi perilaku konkretnya harus ditentukan oleh turunan. Misalnya, semua tile pasti punya aksi saat diinjak, tetapi implementasinya berbeda untuk StreetTile, TaxTile, CardTile, dan lain-lain.

Keuntungan dari penggunaan abstract base class pada kode tersebut adalah desain menjadi lebih terstruktur karena ada kontrak yang jelas untuk semua turunan, mencegah pembuatan object umum yang belum punya perilaku lengkap, serta memudahkan pengembangan fitur baru karena developer cukup membuat turunan baru yang mengikuti antarmuka yang sudah ada.

4.6.2. Aggregation

Aggregation digunakan ketika sebuah object menyimpan referensi atau pointer ke object lain, tetapi object tersebut tidak selalu dimiliki penuh olehnya.

File Flexbox.hpp

```
class Flexbox : public UIElement {
protected:
    std::vector<UIElement*> children;

public:
    Flexbox();
    explicit Flexbox(Rectangle bound);

    float spacing = 10.0f;
    float padding = 10.0f;
    Alignment alignment = Alignment::START;
```

Flexbox membutuhkan Alignment untuk mengatur posisi elemen, misalnya rata kiri, tengah, kanan, atau jarak antar elemen. Relasi ini cocok sebagai aggregation karena Alignment bukan bagian yang wajib mati bersama Flexbox. Satu aturan alignment bisa dipakai ulang oleh beberapa Flexbox.

4.6.3. Composition

File Game.hpp

```
class Game {
private:
    Board board;
```

Game membutuhkan Board sebagai tempat utama permainan berlangsung, misalnya untuk menaruh Tile, menentukan posisi Player, dan menjalankan alur permainan. Relasi ini cocok sebagai composition karena Board merupakan bagian yang bergantung langsung pada Game. Board dibuat ketika Game dimulai dan tidak digunakan lagi ketika Game selesai atau dihapus.

4.6.4. Encapsulation

Encapsulation diterapkan dengan menyimpan data penting di dalam bagian private atau protected, lalu menyediakan akses melalui method publik. Contoh yang cukup jelas ada pada class Player dengan kode di bawah ini :

File Player.hpp

```
class Player
{
private:
    int id;
    Account* account;
    int money;
    int position;
    PlayerStatus status;
    std::vector<PropertyTile*> ownedProperties;
    std::vector<std::unique_ptr<AbilityCard>> handCards;

public:
    void pay(int amount);
    void receive(int amount);
    bool canAfford(int amount) const;
    int getMoney() const { return money; }
    PlayerStatus getStatus() const { return status; }
};
```

Implementasi pay() juga menunjukkan bahwa perubahan state uang tidak dilakukan sembarangan, tetapi melalui validasi.

File Player.cpp

```
void Player::pay(int amount) {
    if (amount > this->money) {
        if (this->ownedProperties.empty()) {
            throw BankruptException(1, "gagal bayar, ga punya properti juga");
        }
        throw InsufficientFundException(amount, this->money);
    }
    this->money -= amount;
}
```

Encapsulation digunakan karena data seperti uang, posisi, status pemain, dan kartu di tangan adalah state penting yang tidak boleh diubah bebas dari luar class. Jika dibuka langsung, logika permainan bisa menjadi tidak konsisten. Selain itu, keuntungan dari penggunaan encapsulation adalah menjaga integritas data object, memusatkan validasi di dalam class, membuat perubahan implementasi internal lebih aman karena kode luar hanya berinteraksi lewat method yang disediakan.

4.6.5. Association

Association adalah relasi ketika dua class saling berhubungan untuk bekerja sama, tetapi tidak selalu menunjukkan kepemilikan penuh. Object satu mengetahui atau memakai object lain agar fungsinya berjalan. Contoh yang jelas ada pada method Game::handleLanding(Player& player).

File Game.cpp

```
void Game::handleLanding(Player& player) {
    Tile* tile = board.getTileByIndex(player.getPosition());
    if (tile != nullptr) {
        tile->onLand(&player, this);
    }
```

```
}
```

Pada kode tersebut, Game berasosiasi dengan Player karena Game perlu menggunakan data dan state milik pemain, misalnya posisi pemain, lalu meneruskan pemain tersebut ke tile yang sedang diinjak. Game dan Player bekerja sama untuk menjalankan logika permainan, tetapi Player tetap merupakan entitas tersendiri yang punya data dan perilakunya sendiri. Alasan penggunaan association pada kasus ini adalah karena alur permainan memang membutuhkan interaksi antar object, bukan sekadar penyimpanan data. Game perlu mengetahui pemain mana yang sedang aktif dan memproses dampak dari tile terhadap pemain tersebut. Keuntungan dari penggunaan association adalah pembagian tanggung jawab lebih jelas, Game mengatur alur permainan, sementara Player tetap fokus pada state pemain, serta interaksi antar komponen menjadi fleksibel dan mudah dikembangkan.

Contoh lain ada pada relasi PropertyTile dengan Player.

File PropertyTile.hpp

```
class PropertyTile : public Tile {
protected:
    Player* owner;
```

File Player.cpp

```
void PropertyTile::setOwner(Player* owner) {
    this->owner = owner;
    status = (owner == nullptr ? BANK : OWNED);
}
```

Kode di atas juga termasuk association karena PropertyTile berhubungan dengan Player sebagai pemilik, tetapi PropertyTile tidak menciptakan object Player. Hubungan ini bisa berubah-ubah selama permainan, misalnya saat jual beli, trade, atau bangkrut.

5. Bonus Yang dikerjakan

5.1. Bonus yang diusulkan oleh spek

5.1.1. GUI

Pada tugas besar ini, kami mengerjakan bonus GUI dengan membangun antarmuka permainan berbasis Raylib. Pendekatan yang kami gunakan adalah memisahkan UI dari core game melalui arsitektur scene-based UI dan Game. Dengan pendekatan ini, antarmuka sudah dapat dijalankan, diuji, dan ditinjau lebih awal meskipun integrasi penuh ke core game belum selesai. Entry point GUI berada pada class Nimonspoli, yang mengatur inisialisasi window, game loop, serta penghubung antara facade dan scene manager.

Alasan kami menggunakan pendekatan ini adalah agar GUI tidak bergantung langsung pada implementasi logic game yang masih berkembang. UI hanya berkomunikasi melalui IGameFacade, sehingga saat core asli selesai, implementasi facade dapat diganti tanpa harus menulis ulang tampilan. Keuntungan pendekatan ini adalah GUI bisa dikembangkan dan dipresentasikan lebih awal, integrasi ke core menjadi lebih mudah karena UI tidak tightly coupled, serta struktur kode lebih modular dan lebih mudah dipelihara.

Implementasi GUI kami terdiri dari dua scene utama, yaitu main menu dan in-game scene. Main menu menyediakan alur awal permainan seperti tombol New Game, Load Demo, Quit, serta modal setup pemain.

File MainMenuScene.cpp

```
MainMenuScene::MainMenuScene(SceneManager* manager, IGameFacade* facade)
    : Scene(manager, facade),
      newGameButton("New Game", kAccent, kText),
      loadDemoButton("Load Demo", kAccentAlt, kText),
      quitButton("Quit", kDanger, kText),
      startGameButton("Start Demo", kAccent, kText) {
```

```
        newGameButton.setOnClick([this]() { showNewGameModal = true; });
        loadDemoButton.setOnClick([this]() {
            gameFacade->loadDemoGame();
            sceneManager->setScene(SceneType::InGame);
        });
    }
```

Pada scene permainan, kami membangun board visual 40 petak, sidebar informasi pemain, daftar aksi, highlight tile terpilih, overlay popup, dan animasi token pemain. Board juga dirender secara dinamis dengan pemetaan indeks ke posisi petak, kemudian setiap tile digambar lengkap dengan pemilik, rumah, hotel, dan efek festival.

File InGameScene.cpp

```
    for (std::size_t i = 0; i < vm.board.size() && i < tileRects.size(); ++i) {
        const TileViewData& tile = vm.board[i];
        Rectangle rect = tileRects[i];
        const bool selected = static_cast<int>(i) == vm.selectedTileIndex;
        DrawRectangleRec(rect, kBoardSurface);
        DrawRectangleLinesEx(rect, 1.0f, Fade(DARKGRAY, 0.35f));
```

```
        if (selected) {
            const float pulse = 0.5f + 0.5f * std::sin(sceneTime * 3.0f);
            DrawRectangleLinesEx(rect, 3.0f + pulse * 2.0f, kAccent);
        }
    }
```

Selain itu, GUI ini didukung komponen reusable seperti Button dan TextField yang memiliki state interaktif sendiri.

File Button.cpp

```
    void Button::update() {
        hovered = CheckCollisionPointRec(GetMousePosition(), boundingBox);
        if (hovered && IsMouseButtonPressed(MOUSE_LEFT_BUTTON)) {
            pressed = true;
        }



        if (pressed && IsMouseButtonReleased(MOUSE_LEFT_BUTTON)) {
```

```
            pressed = false;
            if (hovered && onButtonClick) {
                onButtonClick();
            }
        }
    }
```

File TextField.cpp

```
    void TextField::update() {
        if (IsMouseButtonPressed(MOUSE_LEFT_BUTTON)) {
            isActive = CheckCollisionPointRec(GetMousePosition(), boundingBox);
        }



        if (!isActive) {
            return;
        }
```

```
    int ch = GetCharPressed();
    while (ch > 0) {
        if (ch >= 32 && ch <= 126 && static_cast<int>(text.size()) < maxLength) {
            text.push_back(static_cast<char>(ch));
        }
        ch = GetCharPressed();
    }
}
```

Bonus GUI ini tidak hanya menampilkan tampilan statis, tetapi juga menyediakan alur interaksi yang cukup lengkap: menu awal, setup pemain, perpindahan scene, board interaktif, popup overlay, log, kartu, serta simulasi aksi permainan. Hal ini membuat GUI dapat berfungsi sebagai shell permainan yang meyakinkan sambil menunggu integrasi penuh dengan core logic.

5.2. Bonus Kreasi Mandiri

5.2.1. Account

Salah satu bonus kreasi mandiri yang kami implementasikan adalah fitur akun. Fitur ini memungkinkan setiap pemain memiliki identitas akun berupa username, password, dan score. Dengan adanya sistem akun, permainan tidak hanya berjalan sebagai sesi sementara, tetapi juga memiliki dukungan penyimpanan identitas pemain dan progres nilai antarsesi.

Class utama untuk fitur ini adalah Account. Class ini menyimpan data akun dan menyediakan operasi dasar seperti validasi password, pengambilan nama, pengambilan score, serta penambahan score.

File Account.hpp

```
class Account
{
private:
    std::string username;
    std::string password;
    int score;
public:
    Account(std::string username, std::string password, int score) : username(username), password(password), score(score) {}
    Account(std::string username, std::string password) : username(username), password(password), score(0) {}
    std::string getName() const {
        return username;
    }
    int getScore() const {
        return score;
    }
    bool isPasswordMatch(std::string input) const {
        return input == password;
    }
    void addScore(int num) {
        score += num;
    }
};
```

Selain itu, kami juga menyiapkan `AccountManager` untuk mengelola kumpulan akun, misalnya menambahkan akun baru dan mengambil akun berdasarkan username serta password.

File AccountManager.hpp

```
class AccountManager
{
private:
    vector<Account> listAccount;
public:
    void addAccount(const Account &newAccount);
    Account *getAccount(const string &name, const string &pass);
};
```

Pada implementasi Account, kami memisahkan tanggung jawab menjadi dua bagian, yaitu Account berfokus pada representasi dan perilaku satu akun sementara AccountManager berfokus pada pengelolaan banyak akun. Desain ini dipilih karena lebih modular dan sesuai dengan prinsip OOP. Data akun tidak bercampur langsung dengan logika permainan utama, sehingga fitur akun bisa dikembangkan secara terpisah tanpa mengganggu core game. Keuntungan dari fitur akun ini adalah pemain memiliki identitas yang konsisten, score dapat dicatat dan dikaitkan dengan akun tertentu, sistem lebih siap dikembangkan ke fitur lanjutan seperti login, leaderboard, atau riwayat permainan.

Integrasi fitur akun dengan pemain juga terlihat pada class Player, karena setiap Player menyimpan pointer ke Account.

File Player.hpp

```
class Player
{
private:
    int id;
    Account* account;
```

Lalu username pemain dapat diambil dari akun tersebut:

File Player.cpp

```
std::string Player::getUsername() const {
    if (this->account == nullptr) {
        return "COM" + std::to_string(this->id);
    }
    return this->account->getName();
}
```

6. Pembagian Tugas

Modul (dalam poin spek)	Implementer	Tester
Scene (Bonus)	13524017, 13524029	13524017, 13524029
UIElement (Bonus)	13524017, 13524029	13524017, 13524029
SceneManager (Bonus)	13524017, 13524029	13524017, 13524029
Loader	13524063	13524017, 13524029
Config	13524063	13524017, 13524029
Save/Load	13524063	13524017, 13524029
Board	13524081, 13524029	13524029
Tile	13524081. 13524029	13524029
Player	13524045	13524029
Dice	13524045	13524029
Card	13524045	13524029

Account (Bonus)	13524029,	13524029
GameManager	13524029	13524029
TurnManager	13524029	13524029
AuctionManager	13524029	13524029
BankruptcyManager	13524029	13524029

Lampiran

Lampiran A. Form Asistensi

https://drive.google.com/drive/folders/1uLGrawN_dOCex9ykW_3oj9ILxeFr4vab?usp=drive_link